

```

    <Extensions>
      <uap3:Extension Category="windows.appUriHandler">
        <uap3:AppUriHandler>
          <uap3:Host Name="msn.com" />
        </uap3:AppUriHandler>
      </uap3:Extension>
    </Extensions>
  </Application>
</Applications>

```

The declaration above registers your app to handle links from the specified host. If your website has multiple addresses (for example: m.example.com, www.example.com, and example.com) then add a separate `<uap3:Host Name=... />` entry inside of the `<uap3:AppUriHandler>` for each address.

## Associate your app and website with a JSON file

To ensure that only your app can open content on your website, include your app's package family name in a JSON file located in the web server root, or at the well-known directory on the domain. This signifies that your website gives consent for the listed apps to open content on your site. You can find the package family name in the Packages section in the app manifest designer.

### Important

The JSON file should not have a .json file suffix.

Create a JSON file (without the .json file extension) named **windows-app-web-link** and provide your app's package family name. For example:

JSON

```

[ {
  "packageFamilyName" : "Your app's package family name, e.g
MyApp_9jmtgj1pbbz6e",
  "paths" : [ "*" ],
  "excludePaths" : [ "/news/*", "/blog/*" ]
}]

```

Windows will make an https connection to your website and will look for the corresponding JSON file on your web server.

# Wildcards

The JSON file example above demonstrates the use of wildcards. Wildcards allow you to support a wide variety of links with fewer lines of code. Web-to-app linking supports two types of wildcards in the JSON file:

| Wildcard | Description                   |
|----------|-------------------------------|
| *        | Represents any substring      |
| ?        | Represents a single character |

For example, given `"excludePaths" : [ "/news/*", "/blog/*" ]` in the example above, your app will support all paths that start with your website's address (for example, msn.com), **except** those under `/news/` and `/blog/`. `msn.com/weather.html` will be supported, but not `msn.com/news/topnews.html`.

## Multiple apps

If you have two apps that you would like to link to your website, list both of the application package family names in your **windows-app-web-link** JSON file. Both apps can be supported. The user will be presented with a choice of which is the default link if both are installed. If they want to change the default link later, they can change it in **Settings > Apps for Websites**. Developers can also change the JSON file at any time and see the change as early as the same day but no later than eight days after the update.

JSON

```
[{
  "packageFamilyName": "Your apps's package family name, e.g
MyApp_9jmtgj1pbbz6e",
  "paths": [ "*" ],
  "excludePaths" : [ "/news/*", "/blog/*" ]
},
{
  "packageFamilyName": "Your second app's package family name, for example,
MyApp2_8jmtgj2pbbz6e",
  "paths": [ "/example/*", "/links/*" ]
}]
```

To provide the best experience for your users, use exclude paths to make sure that online-only content is excluded from the supported paths in your JSON file.

Exclude paths are checked first and if there is a match the corresponding page will be opened with the browser instead of the designated app. In the example above, `'/news/*'` includes any pages under that path while `'/news*'` (no forward slash trails 'news') includes any paths under `'news*'` such as `'newslocal/'`, `'newsinternational/'`, and so on.

## Handle links on Activation to link to content

Navigate to **App.xaml.cs** in your app's Visual Studio solution and in **OnActivated()** add handling for linked content. In the following example, the page that is opened in the app depends on the URI path:

CS

```
protected override void OnActivated(IActivatedEventArgs e)
{
    Frame rootFrame = Window.Current.Content as Frame;
    if (rootFrame == null)
    {
        ...
    }

    // Check ActivationKind, Parse URI, and Navigate user to content
    Type deepLinkPageType = typeof(MainPage);
    if (e.Kind == ActivationKind.Protocol)
    {
        var protocolArgs = (ProtocolActivatedEventArgs)e;
        switch (protocolArgs.Uri.AbsolutePath)
        {
            case "/":
                break;
            case "/index.html":
                break;
            case "/sports.html":
                deepLinkPageType = typeof(SportsPage);
                break;
            case "/technology.html":
                deepLinkPageType = typeof(TechnologyPage);
                break;
            case "/business.html":
                deepLinkPageType = typeof(BusinessPage);
                break;
            case "/science.html":
                deepLinkPageType = typeof(SciencePage);
                break;
        }
    }

    if (rootFrame.Content == null)
    {
        // Default navigation
    }
}
```

```

        rootFrame.Navigate(deepLinkPageType, e);
    }

    // Ensure the current window is active
    Window.Current.Activate();
}

```

**Important** Make sure to replace the final `if (rootFrame.Content == null)` logic with `rootFrame.Navigate(deepLinkPageType, e);` as shown in the example above.

## Test it out: Local validation tool

You can test the configuration of your app and website by running the App host registration verifier tool which is available in:

`%windir%\system32\AppHostRegistrationVerifier.exe`

Test the configuration of your app and website by running this tool with the following parameters:

**`AppHostRegistrationVerifier.exe`** *hostname packagefamilyname filepath*

- Hostname: Your website (for example, microsoft.com)
- Package Family Name (PFN): Your app's PFN
- File path: The JSON file for local validation (for example, C:\SomeFolder\windows-app-web-link)

If the tool does not return anything, validation will work on that file when uploaded. If there is an error code, it will not work.

You can enable the following registry key to force path matching for side-loaded apps as part of local validation:

`HKCU\Software\Classes\LocalSettings\Software\Microsoft\Windows\CurrentVersion\AppModel\SystemAppData\YourApp\AppUriHandlers`

Keyname: `ForceValidation` Value: `1`

## Test it: Web validation

Close your application to verify that the app is activated when you click a link. Then, copy the address of one of the supported paths in your website. For example, if your website's address is "msn.com", and one of the support paths is "path1", you would use

`http://msn.com/path1`

Verify that your app is closed. Press **Windows Key + R** to open the **Run** dialog box and paste the link in the window. Your app should launch instead of the web browser.

Additionally, you can test your app by launching it from another app using the [LaunchUriAsync](#) API. You can use this API to test on phones as well.

If you would like to follow the protocol activation logic, set a breakpoint in the **OnActivated** event handler.

## AppUriHandlers tips:

- Make sure to only specify links that your app can handle.
- List all of the hosts that you will support. Note that `www.example.com` and `example.com` are different hosts.
- Users can choose which app they prefer to handle websites in Settings.
- Your JSON file must be uploaded to an https server.
- If you need to change the paths that you wish to support, you can republish your JSON file without republishing your app. Users will see the changes in 1-8 days.
- All sideloaded apps with AppUriHandlers will have validated links for the host on install. You do not need to have a JSON file uploaded to test the feature.
- This feature works whenever your app is a UWP app launched with [LaunchUriAsync](#) or a Windows desktop app launched with [ShellExecuteEx](#). If the URL corresponds to a registered App URI handler, the app will be launched instead of the browser.

## See also

[Web-to-App example project](#) [windows.protocol registration Handle URI Activation Association Launching sample](#) [illustrates how to use the LaunchUriAsync\(\) API.](#)

# Launch an app through file activation

Article • 10/20/2022

This section shows how to set up your app to launch when a file of a certain type is opened.

| Topic                                             | Description                                                                           |
|---------------------------------------------------|---------------------------------------------------------------------------------------|
| <a href="#">Launch the default app for a file</a> | Learn how to launch the default app for a file.                                       |
| <a href="#">Handle file activation</a>            | Learn how to register your app to become the default handler for a certain file type. |

# Launch the default app for a file

Article • 10/20/2022

## Important APIs

- [Windows.System.Launcher.LaunchFileAsync](#)

Learn how to launch the default app for a file. Many apps need to work with files that they can't handle themselves. For example, e-mail apps receive a variety of file types and need a way to launch these files in their default handlers. These steps show how to use the [Windows.System.Launcher](#) API to launch the default handler for a file that your app can't handle itself.

## Get the file object

First, get a [Windows.Storage.StorageFile](#) object for the file.

If the file is included in the package for your app, you can use the [Package.InstalledLocation](#) property to get a [Windows.Storage.StorageFolder](#) object and the [Windows.Storage.StorageFolder.GetFilesAsync](#) method to get the [StorageFile](#) object.

If the file is in a known folder, you can use the properties of the [Windows.Storage.KnownFolders](#) class to get a [StorageFolder](#) and the [GetFilesAsync](#) method to get the [StorageFile](#) object.

## Launch the file

Windows provides several different options for launching the default handler for a file. These options are described in this chart and in the sections that follow.

| Option                                 | Method                                                         | Description                                                                                                                              |
|----------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Default launch                         | <a href="#">LaunchFileAsync(IStorageFile)</a>                  | Launch the specified file with the default handler.                                                                                      |
| Open With launch                       | <a href="#">LaunchFileAsync(IStorageFile, LauncherOptions)</a> | Launch the specified file letting the user pick the handler through the Open With dialog.                                                |
| Launch with a recommended app fallback | <a href="#">LaunchFileAsync(IStorageFile, LauncherOptions)</a> | Launch the specified file with the default handler. If no handler is installed on the system, recommend an app in the store to the user. |

| Option                               | Method                                                                        | Description                                                                                                                                                                                                                                       |
|--------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Launch with a desired remaining view | <a href="#">LaunchFileAsync(IStorageFile, LauncherOptions)</a> (Windows-only) | Launch the specified file with the default handler. Specify a preference to stay on screen after the launch and request a specific window size. <a href="#">LauncherOptions.DesiredRemainingView</a> isn't supported on the mobile device family. |

## Default launch

Call the [Windows.System.Launcher.LaunchFileAsync\(IStorageFile\)](#) method to launch the default app. This example uses the [Windows.Storage.StorageFolder.GetFilesAsync](#) method to launch an image file, test.png, that is included in the app package.

C#

```

async void DefaultLaunch()
{
    // Path to the file in the app package to launch
    string imageFile = @"images\test.png";

    var file = await
Windows.ApplicationModel.Package.Current.InstalledLocation.GetFilesAsync(imageFile);

    if (file != null)
    {
        // Launch the retrieved file
        var success = await Windows.System.Launcher.LaunchFileAsync(file);

        if (success)
        {
            // File launched
        }
        else
        {
            // File launch failed
        }
    }
    else
    {
        // Could not find file
    }
}

```

VB

```

async Sub DefaultLaunch()
    ' Path to the file in the app package to launch

```

```

Dim imageFile = "images\test.png"
Dim file = await
Windows.ApplicationModel.Package.Current.InstalledLocation.GetFilesAsync(imageFile)

If file IsNot Nothing Then
    ' Launch the retrieved file
    Dim success = await Windows.System.Launcher.LaunchFileAsync(file)

    If success Then
        ' File launched
    Else
        ' File launch failed
    End If
Else
    ' Could not find file
End If
End Sub

```

C++/WinRT

```

Windows::Foundation::IAsyncAction MainPage::DefaultLaunch()
{
    auto installFolder{
Windows::ApplicationModel::Package::Current().InstalledLocation() };

    Windows::Storage::StorageFile file{ co_await
installFolder.GetFilesAsync(L"images\\test.png") };

    if (file)
    {
        // Launch the retrieved file
        bool success = co_await
Windows::System::Launcher::LaunchFileAsync(file);
        if (success)
        {
            // File launched
        }
        else
        {
            // File launch failed
        }
    }
    else
    {
        // Could not find file
    }
}

```

C++

```

void MainPage::DefaultLaunch()
{
    auto installFolder = Windows::ApplicationModel::Package::Current->
    InstalledLocation;

    concurrency::task<Windows::Storage::StorageFile^getFileOperation(installFolder->
    GetFileAsync("images\\test.png"));
    getFileOperation.then([](Windows::Storage::StorageFile^ file)
    {
        if (file != nullptr)
        {
            // Launch the retrieved file
            concurrency::task<bool>
            launchFileOperation(Windows::System::Launcher::LaunchFileAsync(file));
            launchFileOperation.then([](bool success)
            {
                if (success)
                {
                    // File launched
                }
                else
                {
                    // File launch failed
                }
            });
        }
        else
        {
            // Could not find file
        }
    });
}

```

## Open With launch

Call the [Windows.System.Launcher.LaunchFileAsync\(IStorageFile, LauncherOptions\)](#) method with [LauncherOptions.DisplayApplicationPicker](#) set to **true** to launch the app that the user selects from the **Open With** dialog box.

We recommend that you use the **Open With** dialog box when the user may want to select an app other than the default for a particular file. For example, if your app allows the user to launch an image file, the default handler will likely be a viewer app. In some cases, the user may want to edit the image instead of viewing it. Use the **Open With** option along with an alternative command in the **AppBar** or in a context menu to let the user bring up the **Open With** dialog and select the editor app in these types of scenarios.

How do you want to open this file?

☒ Use this app for all .png files



Keep using Photos



Microsoft Office 2010



Paint



Windows Photo Viewer

More options

C#

```
async void DefaultLaunch()
{
    // Path to the file in the app package to launch
    string imageFile = @"images\test.png";

    var file = await
Windows.ApplicationModel.Package.Current.InstalledLocation.GetFilesAsync(imageFile);

    if (file != null)
    {
        // Set the option to show the picker
        var options = new Windows.System.LauncherOptions();
        options.DisplayApplicationPicker = true;

        // Launch the retrieved file
        bool success = await Windows.System.Launcher.LaunchFileAsync(file,
options);
        if (success)
        {
            // File launched
        }
        else
        {
            // File launch failed
        }
    }
    else
```

```

{
    // Could not find file
}
}

```

VB

```

async Sub DefaultLaunch()

    ' Path to the file in the app package to launch
    Dim imageFile = "images\test.png"

    Dim file = await
Windows.ApplicationModel.Package.Current.InstalledLocation.GetFilesAsync(imageFile)

    If file IsNot Nothing Then
        ' Set the option to show the picker
        Dim options = Windows.System.LauncherOptions()
        options.DisplayApplicationPicker = True

        ' Launch the retrieved file
        Dim success = await Windows.System.Launcher.LaunchFileAsync(file)

        If success Then
            ' File launched
        Else
            ' File launch failed
        End If
    Else
        ' Could not find file
    End If
End Sub

```

C++/WinRT

```

Windows::Foundation::IAsyncAction MainPage::DefaultLaunch()
{
    auto installFolder{
Windows::ApplicationModel::Package::Current().InstalledLocation() };

    Windows::Storage::StorageFile file{ co_await
installFolder.GetFilesAsync(L"images\\test.png") };

    if (file)
    {
        // Set the option to show the picker
        Windows::System::LauncherOptions launchOptions;
        launchOptions.DisplayApplicationPicker(true);

        // Launch the retrieved file
        bool success = co_await

```

```

Windows::System::Launcher::LaunchFileAsync(file, launchOptions);
    if (success)
    {
        // File launched
    }
    else
    {
        // File launch failed
    }
}
else
{
    // Could not find file
}
}

```

C++

```

void MainPage::DefaultLaunch()
{
    auto installFolder = Windows::ApplicationModel::Package::Current->
    InstalledLocation;

    concurrency::task<Windows::Storage::StorageFile^>
    getFileOperation(installFolder->GetFileAsync("images\\test.png"));
    getFileOperation.then([](Windows::Storage::StorageFile^ file)
    {
        if (file != nullptr)
        {
            // Set the option to show the picker
            auto launchOptions = ref new Windows::System::LauncherOptions();
            launchOptions->DisplayApplicationPicker = true;

            // Launch the retrieved file
            concurrency::task<bool>
            launchFileOperation(Windows::System::Launcher::LaunchFileAsync(file,
            launchOptions));
            launchFileOperation.then([](bool success)
            {
                if (success)
                {
                    // File launched
                }
                else
                {
                    // File launch failed
                }
            });
        }
        else
        {
            // Could not find file
        }
    }
    }
}

```

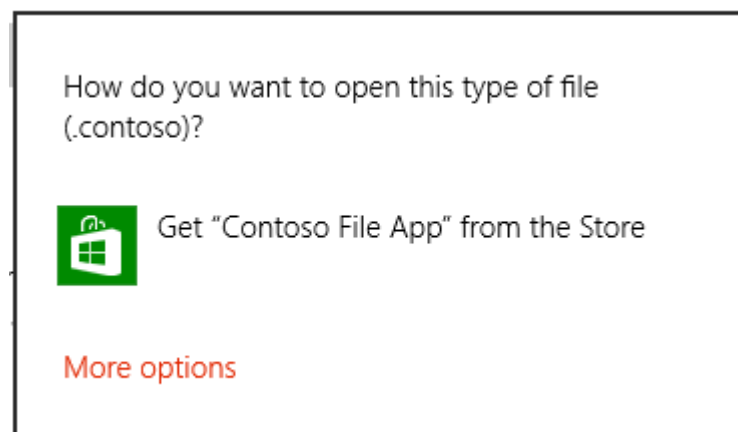
```
});  
}
```

## Launch with a recommended app fallback

In some cases the user may not have an app installed to handle the file that you are launching. By default, Windows will handle these cases by providing the user with a link to search for an appropriate app on the store. If you would like to give the user a specific recommendation for which app to acquire in this scenario, you may do so by passing that recommendation along with the file that you are launching. To do this, call the [Windows.System.Launcher.launchFileAsync\(IStorageFile, LauncherOptions\)](#) method with [LauncherOptions.PreferredApplicationPackageFamilyName](#) set to the package family name of the app in the Store that you want to recommend. Then, set the [LauncherOptions.PreferredApplicationDisplayName](#) to the name of that app. Windows will use this information to replace the general option to search for an app in the store with a specific option to acquire the recommended app from the Store.

### ⓘ Note

You must set both of these options to recommend an app. Setting one without the other will result in a failure.



C#

```
async void DefaultLaunch()  
{  
    // Path to the file in the app package to launch  
    string imageFile = @"images\test.contoso";  
  
    // Get the image file from the package's image directory  
    var file = await  
Windows.ApplicationModel.Package.Current.InstalledLocation.GetFilesAsync(imageFile);
```

```

if (file != null)
{
    // Set the recommended app
    var options = new Windows.System.LauncherOptions();
    options.PreferredApplicationPackageFamilyName =
"Contoso.FileApp_8wknc82po1e";
    options.PreferredApplicationDisplayName = "Contoso File App";

    // Launch the retrieved file pass in the recommended app
    // in case the user has no apps installed to handle the file
    bool success = await Windows.System.Launcher.LaunchFileAsync(file,
options);
    if (success)
    {
        // File launched
    }
    else
    {
        // File launch failed
    }
}
else
{
    // Could not find file
}
}

```

VB

```

async Sub DefaultLaunch()

    ' Path to the file in the app package to launch
    Dim imageFile = "images\test.contoso"

    ' Get the image file from the package's image directory
    Dim file = await
Windows.ApplicationModel.Package.Current.InstalledLocation.GetFilesAsync(imag
eFile)

    If file IsNot Nothing Then
        ' Set the recommended app
        Dim options = Windows.System.LauncherOptions()
        options.PreferredApplicationPackageFamilyName =
"Contoso.FileApp_8wknc82po1e";
        options.PreferredApplicationDisplayName = "Contoso File App";

        ' Launch the retrieved file pass in the recommended app
        ' in case the user has no apps installed to handle the file
        Dim success = await Windows.System.Launcher.LaunchFileAsync(file)

        If success Then
            ' File launched
        Else

```

```

        ' File launch failed
    End If
Else
    ' Could not find file
End If
End Sub

```

C++/WinRT

```

Windows::Foundation::IAsyncAction MainPage::DefaultLaunch()
{
    auto installFolder{
Windows::ApplicationModel::Package::Current().InstalledLocation() };

    Windows::Storage::StorageFile file{ co_await
installFolder.GetFilesAsync(L"images\\test.png") };

    if (file)
    {
        // Set the recommended app
        Windows::System::LauncherOptions launchOptions;

        launchOptions.PreferredApplicationPackageFamilyName(L"Contoso.FileApp_8wknc8
2po1e");
        launchOptions.PreferredApplicationDisplayName(L"Contoso File App");

        // Launch the retrieved file, and pass in the recommended app
        // in case the user has no apps installed to handle the file.
        bool success = co_await
Windows::System::Launcher::LaunchFileAsync(file, launchOptions);
        if (success)
        {
            // File launched
        }
        else
        {
            // File launch failed
        }
    }
    else
    {
        // Could not find file
    }
}

```

C++

```

void MainPage::DefaultLaunch()
{
    auto installFolder = Windows::ApplicationModel::Package::Current-
>InstalledLocation;

```

```

        concurrency::task<Windows::Storage::StorageFile^>
        getFileOperation(installFolder->GetFileAsync("images\\test.contoso"));
        getFileOperation.then([](Windows::Storage::StorageFile^ file)
        {
            if (file != nullptr)
            {
                // Set the recommended app
                auto launchOptions = ref new Windows::System::LauncherOptions();
                launchOptions->PreferredApplicationPackageFamilyName =
                "Contoso.FileApp_8wknc82po1e";
                launchOptions->PreferredApplicationDisplayName = "Contoso File
                App";

                // Launch the retrieved file pass, and in the recommended app
                // in case the user has no apps installed to handle the file.
                concurrency::task<bool>
                launchFileOperation(Windows::System::Launcher::LaunchFileAsync(file,
                launchOptions));
                launchFileOperation.then([](bool success)
                {
                    if (success)
                    {
                        // File launched
                    }
                    else
                    {
                        // File launch failed
                    }
                });
            }
            else
            {
                // Could not find file
            }
        });
    }
}

```

## Launch with a Desired Remaining View (Windows-only)

Source apps that call [LaunchFileAsync](#) can request that they remain on screen after a file launch. By default, Windows attempts to share all available space equally between the source app and the target app that handles the file. Source apps can use the [DesiredRemainingView](#) property to indicate to the operating system that they prefer their app window to take up more or less of the available space. **DesiredRemainingView** can also be used to indicate that the source app does not need to remain on screen after the file launch and can be completely replaced by the target app. This property only specifies the preferred window size of the calling app. It doesn't specify the behavior of other apps that may happen to also be on screen at the same time.

## ⓘ Note

Windows takes into account multiple different factors when it determines the source app's final window size, for example, the preference of the source app, the number of apps on screen, the screen orientation, and so on. By setting **DesiredRemainingView**, you aren't guaranteed a specific windowing behavior for the source app.

**\*\*Mobile device family:** **\*\*[LauncherOptions.DesiredRemainingView](#)** isn't supported on the mobile device family.

C#

```
async void DefaultLaunch()
{
    // Path to the file in the app package to launch
    string imageFile = @"images\test.png";

    var file = await
Windows.ApplicationModel.Package.Current.InstalledLocation.GetFilesAsync(imageFile);

    if (file != null)
    {
        // Set the desired remaining view
        var options = new Windows.System.LauncherOptions();
        options.DesiredRemainingView =
Windows.UI.ViewManagement.ViewSizePreference.UseLess;

        // Launch the retrieved file
        bool success = await Windows.System.Launcher.LaunchFileAsync(file,
options);
        if (success)
        {
            // File launched
        }
        else
        {
            // File launch failed
        }
    }
    else
    {
        // Could not find file
    }
}
```

```

Windows::Foundation::IAsyncAction MainPage::DefaultLaunch()
{
    auto installFolder{
Windows::ApplicationModel::Package::Current().InstalledLocation() };

    Windows::Storage::StorageFile file{ co_await
installFolder.GetFilesAsync(L"images\\test.png") };

    if (file)
    {
        // Set the desired remaining view.
        Windows::System::LauncherOptions launchOptions;

launchOptions.DesiredRemainingView(Windows::UI::ViewManagement::ViewSizePref
erence::UseLess);

        // Launch the retrieved file.
        bool success = co_await
Windows::System::Launcher::LaunchFileAsync(file, launchOptions);
        if (success)
        {
            // File launched
        }
        else
        {
            // File launch failed
        }
    }
    else
    {
        // Could not find file
    }
}

```

C++

```

void MainPage::DefaultLaunch()
{
    auto installFolder = Windows::ApplicationModel::Package::Current-
>InstalledLocation;

    concurrency::task<Windows::Storage::StorageFile^>
getFileOperation(installFolder->GetFilesAsync("images\\test.png"));
getFileOperation.then([](Windows::Storage::StorageFile^ file)
{
    if (file != nullptr)
    {
        // Set the desired remaining view.
        auto launchOptions = ref new Windows::System::LauncherOptions();
        launchOptions->DesiredRemainingView =
Windows::UI::ViewManagement::ViewSizePreference::UseLess;

```

```

        // Launch the retrieved file.
        concurrency::task<bool>
launchFileOperation(Windows::System::Launcher::LaunchFileAsync(file,
launchOptions));
        launchFileOperation.then([](bool success)
        {
            if (success)
            {
                // File launched
            }
            else
            {
                // File launch failed
            }
        });
    }
    else
    {
        // Could not find file
    }
});
}

```

## Remarks

Your app can't select the app that is launched. The user determines which app is launched. The user can select either a Universal Windows Platform (UWP) app or a Windows desktop app.

When launching a file, your app must be the foreground app, that is, it must be visible to the user. This requirement helps ensure that the user remains in control. To meet this requirement, make sure that you tie all file launches directly to the UI of your app. Most likely, the user must always take some action to initiate a file launch.

You can't launch file types that contain code or script if they are executed automatically by the operating system, such as, .exe, .msi, and .js files. This restriction protects users from potentially malicious files that could modify the operating system. You can use this method to launch file types that can contain script if they are executed by an app that isolates the script, such as, .docx files. Apps like Microsoft Word keep the script in .docx files from modifying the operating system.

If you try to launch a restricted file type, the launch will fail and your error callback will be invoked. If your app handles many different types of files and you expect that you will hit this error, we recommend that you provide a fallback experience to your user. For example, you could give the user an option to save the file to the desktop, and they could open it there.

# Related topics

## Tasks

- [Launch the default app for a URI](#)
- [Handle file activation](#)

## Guidelines

- [Guidelines for file types and URIs](#)

## Reference

- [Windows.Storage.StorageFile](#)
- [Windows.System.Launcher.LaunchFileAsync](#)

# Handle file activation

Article • 10/20/2022

## Important APIs

- [Windows.ApplicationModel.Activation.FileActivatedEventArgs](#)
- [Windows.UI.Xaml.Application.OnFileActivated](#)

Your app can register to become the default handler for a certain file type. Both Windows desktop applications and Universal Windows Platform (UWP) apps can register to be a default file handler. If the user chooses your app as the default handler for a certain file type, your app will be activated when that type of file is launched.

We recommend that you only register for a file type if you expect to handle all file launches for that type of file. If your app only needs to use the file type internally, then you don't need to register to be the default handler. If you do choose to register for a file type, you must provide the end user with the functionality that is expected when your app is activated for that file type. For example, a picture viewer app may register to display a .jpg file. For more info on file associations, see [Guidelines for file types and URIs](#).

These steps show how to register for a custom file type, .alsdk, and how to activate your app when the user launches an .alsdk file.

**Note** In UWP apps, certain URIs and file extensions are reserved for use by built-in apps and the operating system. Attempts to register your app with a reserved URI or file extension will be ignored. For more information, see [Reserved file and URI scheme names](#).

## Step 1: Specify the extension point in the package manifest

The app receives activation events only for the file extensions listed in the package manifest. Here's how you indicate that your app handles the files with the `.alsdk` extension.

1. In the **Solution Explorer**, double-click package.appxmanifest to open the manifest designer. Select the **Declarations** tab and in the **Available Declarations** drop-down, select **File Type Associations** and then click **Add**. See [Programmatic Identifiers](#) for more details of identifiers used by file associations.

Here is a brief description of each of the fields that you may fill in the manifest designer:

| Field        | Description                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Display Name | Specify the display name for a group of file types. The display name is used to identify the file type in the <a href="#">Set Default Programs</a> on the <b>Control Panel</b> .                                                                                                                                                                                                                                                  |
| Logo         | Specify the logo that is used to identify the file type on the desktop and in the <a href="#">Set Default Programs</a> on the <b>Control Panel</b> . If no Logo is specified, the application's small logo is used.                                                                                                                                                                                                               |
| Info Tip     | Specify the <a href="#">info tip</a> for a group of file types. This tool tip text appears when the user hovers on the icon for a file of this type.                                                                                                                                                                                                                                                                              |
| Name         | Choose a name for a group of file types that share the same display name, logo, info tip, and edit flags. Choose a group name that can stay the same across app updates.<br><b>Note</b> The Name must be in all lower case letters.                                                                                                                                                                                               |
| Content Type | Specify the MIME content type, such as <b>image/jpeg</b> , for a particular file type.<br><b>Important Note about allowed content types:</b> Here is an alphabetic list of MIME content types that you cannot enter into the package manifest because they are either reserved or forbidden: <b>application/force-download</b> , <b>application/octet-stream</b> , <b>application/unknown</b> , <b>application/x-msdownload</b> . |
| File type    | Specify the file type to register for, preceded by a period, for example, ".jpeg".<br><b>Reserved and forbidden file types:</b> See <a href="#">Reserved URI scheme names and file types</a> for an alphabetic list of file types for built-in apps that you can't register for your UWP apps because they are either reserved or forbidden.                                                                                      |

2. Enter `alsdk` as the **Name**.
3. Enter `.alsdk` as the **File Type**.
4. Enter "images\icon.png" as the Logo.
5. Press Ctrl+S to save the change to package.appxmanifest.

The steps above add an [Extension](#) element like this one to the package manifest. The **windows.fileTypeAssociation** category indicates that the app handles files with the `.alsdk` extension.

XML

```
<Extensions>
  <uap:Extension Category="windows.fileTypeAssociation">
    <uap:FileTypeAssociation Name="alsdk">
      <uap:Logo>images\icon.png</uap:Logo>
      <uap:SupportedFileTypes>
        <uap:FileType>.alsdk</uap:FileType>
      </uap:SupportedFileTypes>
    </uap:FileTypeAssociation>
  </uap:Extension>
</Extensions>
```

```
</uap:Extension>
</Extensions>
```

## Step 2: Add the proper icons

Apps that become the default for a file type have their icons displayed in various places throughout the system. For example, these icons are shown in:

- Windows Explorer Items View, context menus, and the Ribbon
- Default programs Control Panel
- File picker
- Search results on the Start screen

Include a 44x44 icon with your project so that your logo can appear in those locations. Match the look of the app tile logo and use your app's background color rather than making the icon transparent. Have the logo extend to the edge without padding it. Test your icons on white backgrounds. See [Guidelines for tile and icon assets](#) for more details about icons.

## Step 3: Handle the activated event

The [OnFileActivated](#) event handler receives all file activation events.

C#

```
protected override void OnFileActivated(FileActivatedEventArgs args)
{
    // TODO: Handle file activation
    // The number of files received is args.Files.Count
    // The name of the first file is args.Files[0].Name
}
```

### ⓘ Note

When launched via File Contract, make sure that Back button takes the user back to the screen that launched the app and not to the app's previous content.

We recommend that you create a new XAML **Frame** for each activation event that opens a new page. That way, the navigation backstack for the new XAML Frame doesn't contain any previous content that the app might have on the current window when suspended. If you decide to use a single XAML **Frame** for Launch and for File Contracts,

then you should clear the pages in the **Frame**'s navigation journal before navigating to a new page.

When your app is launched via File activation, you should consider including UI that allows the user to go back to the top page of the app.

## Remarks

The files that you receive could come from an untrusted source. We recommend that you validate the content of a file before taking action on it.

## Related topics

### Complete example

- [Association launching sample](#) 

### Concepts

- [Default Programs](#)
- [File Type and Protocol Associations Model](#)

### Tasks

- [Launch the default app for a file](#)
- [Handle URI activation](#)

### Guidelines

- [Guidelines for file types and URIs](#)

### Reference

- [Windows.ApplicationModel.Activation.FileActivatedEventArgs](#)
- [Windows.UI.Xaml.Application.OnFileActivated](#)

# Auto-launching with AutoPlay

Article • 10/20/2022

You can use **AutoPlay** to provide your app as an option when a user connects a device to their PC. This includes non-volume devices such as a camera or media player, or volume devices such as a USB thumb drive, SD card, or DVD. You can also use **AutoPlay** to offer your app as an option when users share files between two PCs by using proximity (tapping).

## ⓘ Note

If you are a device manufacturer and you want to associate your [Microsoft Store device app](#) as an **AutoPlay** handler for your device, you can identify that app in the device metadata. For more info, see [AutoPlay for Microsoft Store device apps](#).

## Register for AutoPlay content

You can register apps as options for **AutoPlay** content events. **AutoPlay** content events are raised when a volume device such as a camera memory card, thumb drive, or DVD is inserted into the PC. Here we show how to identify your app as an **AutoPlay** option when a volume device from a camera is inserted.

In this tutorial, you created an app that displays image files or copies them to Pictures. You registered the app for the AutoPlay **ShowPicturesOnArrival** content event.

AutoPlay also raises content events for content shared between PCs using proximity (tapping). You can use the steps and code in this section to handle files that are shared between PCs that use proximity. The following table lists the AutoPlay content events that are available for sharing content by using proximity.

 Expand table

| Action         | AutoPlay content event  |
|----------------|-------------------------|
| Sharing music  | PlayMusicFilesOnArrival |
| Sharing videos | PlayVideoFilesOnArrival |

When files are shared by using proximity, the **Files** property of the **FileActivatedEventArgs** object contains a reference to a root folder that contains all of the shared files.

## Step 1: Create a new project and add AutoPlay declarations

1. Open Microsoft Visual Studio and select **New Project** from the **File** menu. In the **Visual C#** section, under **Windows**, select **Blank App (Universal Windows)**. Name the app **AutoPlayDisplayOrCopyImages** and click **OK**.
2. Open the Package.appxmanifest file and select the **Capabilities** tab. Select the **Removable Storage** and **Pictures Library** capabilities. This gives the app access to removable storage devices for camera memory, and access to local pictures.
3. In the manifest file, select the **Declarations** tab. In the **Available Declarations** drop-down list, select **AutoPlay Content** and click **Add**. Select the new **AutoPlay Content** item that was added to the **Supported Declarations** list.
4. An **AutoPlay Content** declaration identifies your app as an option when AutoPlay raises a content event. The event is based on the content of a volume device such as a DVD or a thumb drive. AutoPlay examines the content of the volume device and determines which content event to raise. If the root of the volume contains a DCIM, AVCHD, or PRIVATE\ACHD folder, or if a user has enabled **Choose what to do with each type of media** in the AutoPlay Control Panel and pictures are found in the root of the volume, then AutoPlay raises the **ShowPicturesOnArrival** event. In the **Launch Actions** section, enter the values from Table 1 below for the first launch action.
5. In the **Launch Actions** section for the **AutoPlay Content** item, click **Add New** to add a second launch action. Enter the values in Table 2 below for the second launch action.
6. In the **Available Declarations** drop-down list, select **File Type Associations** and click **Add**. In the Properties of the new **File Type Associations** declaration, set the **Display Name** field to **AutoPlay Copy or Show Images** and the **Name** field to **image\_association1**. In the **Supported File Types** section, click **Add New**. Set the **File Type** field to **.jpg**. In the **Supported File Types** section, set the **File Type** field of the new file association to **.png**. For content events, AutoPlay filters out any file types that are not explicitly associated with your app.
7. Save and close the manifest file.

Table 1

 Expand table

| Setting             | Value         |
|---------------------|---------------|
| Verb                | show          |
| Action Display Name | Show Pictures |

| Setting       | Value                 |
|---------------|-----------------------|
| Content Event | ShowPicturesOnArrival |

The **Action Display Name** setting identifies the string that AutoPlay displays for your app. The **Verb** setting identifies a value that is passed to your app for the selected option. You can specify multiple launch actions for an AutoPlay event and use the **Verb** setting to determine which option a user has selected for your app. You can tell which option the user selected by checking the **verb** property of the startup event arguments passed to your app. You can use any value for the **Verb** setting except, **open**, which is reserved.

Table 2

[Expand table](#)

| Setting             | Value                      |
|---------------------|----------------------------|
| Verb                | copy                       |
| Action Display Name | Copy Pictures Into Library |
| Content Event       | ShowPicturesOnArrival      |

## Step 2: Add XAML UI

Open the MainPage.xaml file and add the following XAML to the default <Grid> section.

XAML

```
<TextBlock FontSize="18">File List</TextBlock>
<TextBlock x:Name="FilesBlock" HorizontalAlignment="Left"
TextWrapping="Wrap"
VerticalAlignment="Top" Margin="0,20,0,0" Height="280"
Width="240" />
<Canvas x:Name="FilesCanvas" HorizontalAlignment="Left"
VerticalAlignment="Top"
Margin="260,20,0,0" Height="280" Width="100"/>
```

## Step 3: Add initialization code

The code in this step checks the verb value in the **Verb** property, which is one of the startup arguments passed to the app during the **OnFileActivated** event. The code then calls a method related to the option that the user selected. For the camera memory

event, AutoPlay passes the root folder of the camera storage to the app. You can retrieve this folder from the first element of the **Files** property.

Open the App.xaml.cs file and add the following code to the **App** class.

```
cs

protected override void OnFileActivated(FileActivatedEventArgs args)
{
    if (args.Verb == "show")
    {
        Frame rootFrame = (Frame)Window.Current.Content;
        MainPage page = (MainPage)rootFrame.Content;

        // Call DisplayImages with root folder from camera storage.
        page.DisplayImages((Windows.Storage.StorageFolder)args.Files[0]);
    }

    if (args.Verb == "copy")
    {
        Frame rootFrame = (Frame)Window.Current.Content;
        MainPage page = (MainPage)rootFrame.Content;

        // Call CopyImages with root folder from camera storage.
        page.CopyImages((Windows.Storage.StorageFolder)args.Files[0]);
    }

    base.OnFileActivated(args);
}
```

**Note** The `DisplayImages` and `CopyImages` methods are added in the following steps.

## Step 4: Add code to display images

In the MainPage.xaml.cs file add the following code to the **MainPage** class.

```
cs

async internal void DisplayImages(Windows.Storage.StorageFolder rootFolder)
{
    // Display images from first folder in root\DCIM.
    var dcimFolder = await rootFolder.GetFolderAsync("DCIM");
    var folderList = await dcimFolder.GetFoldersAsync();
    var cameraFolder = folderList[0];
    var fileList = await cameraFolder.GetFilesAsync();
    for (int i = 0; i < fileList.Count; i++)
    {
        var file = (Windows.Storage.StorageFile)fileList[i];
        WriteMessageText(file.Name + "\n");
        DisplayImage(file, i);
    }
}
```

```

    }
}

async private void DisplayImage(Windows.Storage.IStorageItem file, int
index)
{
    try
    {
        var sFile = (Windows.Storage.StorageFile)file;
        Windows.Storage.Streams.IRandomAccessStream imageStream =
            await sFile.OpenAsync(Windows.Storage.FileAccessMode.Read);
        Windows.UI.Xaml.Media.Imaging.BitmapImage imageBitmap =
            new Windows.UI.Xaml.Media.Imaging.BitmapImage();
        imageBitmap.SetSource(imageStream);
        var element = new Image();
        element.Source = imageBitmap;
        element.Height = 100;
        Thickness margin = new Thickness();
        margin.Top = index * 100;
        element.Margin = margin;
        FilesCanvas.Children.Add(element);
    }
    catch (Exception e)
    {
        WriteMessageText(e.Message + "\n");
    }
}

// Write a message to MessageBlock on the UI thread.
private Windows.UI.Core.CoreDispatcher messageDispatcher =
Window.Current.CoreWindow.Dispatcher;

private async void WriteMessageText(string message, bool overwrite = false)
{
    await
messageDispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        if (overwrite)
            FilesBlock.Text = message;
        else
            FilesBlock.Text += message;
    });
}

```

## Step 5: Add code to copy images

In the MainPage.xaml.cs file add the following code to the **MainPage** class.

```

async internal void CopyImages(Windows.Storage.StorageFolder rootFolder)
{
    // Copy images from first folder in root\DCIM.
    var dcimFolder = await rootFolder.GetFolderAsync("DCIM");
    var folderList = await dcimFolder.GetFoldersAsync();
    var cameraFolder = folderList[0];
    var fileList = await cameraFolder.GetFilesAsync();

    try
    {
        var folderName = "Images " + DateTime.Now.ToString("yyyy-MM-dd
HHmmss");
        Windows.Storage.StorageFolder imageFolder = await
Windows.Storage.KnownFolders.PicturesLibrary.CreateFolderAsync(folderName);

        foreach (Windows.Storage.IStorageItem file in fileList)
        {
            CopyImage(file, imageFolder);
        }
    }
    catch (Exception e)
    {
        WriteMessageText("Failed to copy images.\n" + e.Message + "\n");
    }
}

async internal void CopyImage(Windows.Storage.IStorageItem file,
                               Windows.Storage.StorageFolder imageFolder)
{
    try
    {
        Windows.Storage.StorageFile sFile =
(Windows.Storage.StorageFile)file;
        await sFile.CopyAsync(imageFolder, sFile.Name);
        WriteMessageText(sFile.Name + " copied.\n");
    }
    catch (Exception e)
    {
        WriteMessageText("Failed to copy file.\n" + e.Message + "\n");
    }
}

```

## Step 6: Build and run the app

1. Press F5 to build and deploy the app (in debug mode).
2. To run your app, insert a camera memory card or another storage device from a camera into your PC. Then, select one of the content event options that you specified in your package.appxmanifest file from the AutoPlay list of options. This sample code only displays or copies pictures in the DCIM folder of a camera

memory card. If your camera memory card stores pictures in an AVCHD or PRIVATE\ACHD folder, you will need to update the code accordingly.

#### ⓘ Note

If you don't have a camera memory card, you can use a flash drive if it has a folder named **DCIM** in the root and if the DCIM folder has a subfolder that contains images.

## Register for an AutoPlay device

You can register apps as options for **AutoPlay** device events. **AutoPlay** device events are raised when a device is connected to a PC.

Here we show how to identify your app as an **AutoPlay** option when a camera is connected to a PC. The app registers as a handler for the **WPD\ImageSourceAutoPlay** event. This is a common event that the Windows Portable Device (WPD) system raises when cameras and other imaging devices notify it that they are an ImageSource using MTP. For more info, see [Windows Portable Devices](#).

**Important** The [Windows.Devices.Portable.StorageDevice](#) APIs are part of the [desktop device family](#). Apps can use these APIs only on Windows 10 devices in the desktop device family, such as PCs.

## Step 1: Create another new project and add AutoPlay declarations

1. Open Visual Studio and select **New Project** from the **File** menu. In the **Visual C#** section, under **Windows**, select **Blank App (Universal Windows)**. Name the app **AutoPlayDevice\_Camera** and click **OK**.
2. Open the Package.appxmanifest file and select the **Capabilities** tab. Select the **Removable Storage** capability. This gives the app access to the data on the camera as a removable storage volume device.
3. In the manifest file, select the **Declarations** tab. In the **Available Declarations** drop-down list, select **AutoPlay Device** and click **Add**. Select the new **AutoPlay Device** item that was added to the **Supported Declarations** list.
4. An **AutoPlay Device** declaration identifies your app as an option when AutoPlay raises a device event for known events. In the **Launch Actions** section, enter the values in the table below for the first launch action.

5. In the **Available Declarations** drop-down list, select **File Type Associations** and click **Add**. In the Properties of the new **File Type Associations** declaration, set the **Display Name** field to **Show Images from Camera** and the **Name** field to **camera\_association1**. In the **Supported File Types** section, click **Add New** (if needed). Set the **File Type** field to **.jpg**. In the **Supported File Types** section, click **Add New** again. Set the **File Type** field of the new file association to **.png**. For content events, AutoPlay filters out any file types that are not explicitly associated with your app.
6. Save and close the manifest file.

 Expand table

| Setting             | Value           |
|---------------------|-----------------|
| Verb                | show            |
| Action Display Name | Show Pictures   |
| Content Event       | WPD\ImageSource |

The **Action Display Name** setting identifies the string that AutoPlay displays for your app. The **Verb** setting identifies a value that is passed to your app for the selected option. You can specify multiple launch actions for an AutoPlay event and use the **Verb** setting to determine which option a user has selected for your app. You can tell which option the user selected by checking the **verb** property of the startup event arguments passed to your app. You can use any value for the **Verb** setting except, **open**, which is reserved. For an example of using multiple verbs in a single app, see [Register for AutoPlay content](#).

## Step 2: Add assembly reference for the desktop extensions

The APIs required to access storage on a Windows Portable Device, [Windows.Devices.Portable.StorageDevice](#), are part of the [desktop device family](#). This means a special assembly is required to use the APIs and those calls will only work on a device in the desktop device family (such as a PC).

1. In **Solution Explorer**, right click on **References** and then **Add Reference....**
2. Expand **Universal Windows** and click **Extensions**.
3. Then select **Windows Desktop Extensions for the UWP** and click **OK**.

## Step 3: Add XAML UI

Open the MainPage.xaml file and add the following XAML to the default <Grid> section.

XAML

```
<StackPanel Orientation="Vertical" Margin="10,0,-10,0">
    <TextBlock FontSize="24">Device Information</TextBlock>
    <StackPanel Orientation="Horizontal">
        <TextBlock x:Name="DeviceInfoTextBlock" FontSize="18" Height="400"
Width="400" VerticalAlignment="Top" />
        <ListView x:Name="ImagesList" HorizontalAlignment="Left"
Height="400" VerticalAlignment="Top" Width="400">
            <ListView.ItemTemplate>
                <DataTemplate>
                    <StackPanel Orientation="Vertical">
                        <Image Source="{Binding Path=Source}" />
                        <TextBlock Text="{Binding Path=Name}" />
                    </StackPanel>
                </DataTemplate>
            </ListView.ItemTemplate>
            <ListView.ItemsPanel>
                <ItemsPanelTemplate>
                    <WrapGrid Orientation="Horizontal" ItemHeight="100"
ItemWidth="120"></WrapGrid>
                </ItemsPanelTemplate>
            </ListView.ItemsPanel>
        </ListView>
    </StackPanel>
</StackPanel>
```

## Step 4: Add activation code

The code in this step references the camera as a [StorageDevice](#) by passing the device information Id of the camera to the [FromId](#) method. The device information Id of the camera is obtained by first casting the event arguments as [DeviceActivatedEventArgs](#), and then getting the value from the [DeviceInformationId](#) property.

Open the App.xaml.cs file and add the following code to the **App** class.

CS

```
protected override void OnActivated(IActivatedEventArgs args)
{
    if (args.Kind == ActivationKind.Device)
    {
        Frame rootFrame = null;
        // Ensure that the current page exists and is activated
        if (Window.Current.Content == null)
        {
            rootFrame = new Frame();
            rootFrame.Navigate(typeof(MainPage));
        }
    }
}
```

```

        Window.Current.Content = rootFrame;
    }
    else
    {
        rootFrame = Window.Current.Content as Frame;
    }
    Window.Current.Activate();

    // Make sure the necessary APIs are present on the device
    bool storageDeviceAPIPresent =

Windows.Foundation.Metadata.ApiInformation.IsTypePresent("Windows.Devices.Portable.StorageDevice");

    if (storageDeviceAPIPresent)
    {
        // Reference the current page as type MainPage
        var mPage = rootFrame.Content as MainPage;

        // Cast the activated event args as DeviceActivatedEventArgs and
show images
        var deviceArgs = args as DeviceActivatedEventArgs;
        if (deviceArgs != null)
        {

mPage.ShowImages(Windows.Devices.Portable.StorageDevice.FromId(deviceArgs.DeviceInformationId));
        }
    }
    else
    {
        // Handle case where APIs are not present (when the device is not
part of the desktop device family)
    }

}

base.OnActivated(args);
}

```

### ⓘ Note

The `ShowImages` method is added in the following step.

## Step 5: Add code to display device information

You can obtain information about the camera from the properties of the [StorageDevice](#) class. The code in this step displays the device name and other info to the user when the

app runs. The code then calls the `GetImageList` and `GetThumbnail` methods, which you will add in the next step, to display thumbnails of the images stored on the camera

In the `MainPage.xaml.cs` file, add the following code to the **MainPage** class.

CS

```
private Windows.Storage.StorageFolder rootFolder;

internal async void ShowImages(Windows.Storage.StorageFolder folder)
{
    DeviceInfoTextBlock.Text = "Display Name = " + folder.DisplayName +
    "\n";
    DeviceInfoTextBlock.Text += "Display Type = " + folder.DisplayType +
    "\n";
    DeviceInfoTextBlock.Text += "FolderRelativeId = " +
    folder.FolderRelativeId + "\n";

    // Reference first folder of the device as the root
    rootFolder = (await folder.GetFoldersAsync())[0];
    var imageList = await GetImageList(rootFolder);

    foreach (Windows.Storage.StorageFile img in imageList)
    {
        ImagesList.Items.Add(await GetThumbnail(img));
    }
}
```

#### ⓘ Note

The `GetImageList` and `GetThumbnail` methods are added in the following step.

## Step 6: Add code to display images

The code in this step displays thumbnails of the images stored on the camera. The code makes asynchronous calls to the camera to get the thumbnail image. However, the next asynchronous call doesn't occur until the previous asynchronous call completes. This ensures that only one request is made to the camera at a time.

In the `MainPage.xaml.cs` file, add the following code to the **MainPage** class.

CS

```
async private System.Threading.Tasks.Task<List<Windows.Storage.StorageFile>>
GetImageList(Windows.Storage.StorageFolder folder)
{
    var result = await folder.GetFilesAsync();
```

```

var subFolders = await folder.GetFoldersAsync();
foreach (Windows.Storage.StorageFolder f in subFolders)
    result = result.Union(await GetImageList(f)).ToList();

return (from f in result orderby f.Name select f).ToList();
}

async private System.Threading.Tasks.Task<Image>
GetThumbnail(Windows.Storage.StorageFile img)
{
    // Get the thumbnail to display
    var thumbnail = await
img.GetThumbnailAsync(Windows.Storage.FileProperties.ThumbnailMode.SingleItem,
                                100,
Windows.Storage.FileProperties.ThumbnailOptions.UseCurrentScale);

    // Create a XAML Image object bind to on the display page
    var result = new Image();
    result.Height = thumbnail.OriginalHeight;
    result.Width = thumbnail.OriginalWidth;
    result.Name = img.Name;
    var imageBitmap = new Windows.UI.Xaml.Media.Imaging.BitmapImage();
    imageBitmap.SetSource(thumbnail);
    result.Source = imageBitmap;

    return result;
}

```

## Step 7: Build and run the app

1. Press F5 to build and deploy the app (in debug mode).
2. To run your app, connect a camera to your machine. Then select the app from the AutoPlay list of options.

### ⓘ Note

Not all cameras advertise for the **WPD\ImageSource** AutoPlay device event.

## Configure removable storage

You can identify a volume device such as a memory card or thumb drive as an **AutoPlay** device when the volume device is connected to a PC. This is especially useful when you want to associate a specific app for **AutoPlay** to present to the user for your volume device.

Here we show how to identify your volume device as an **AutoPlay** device.

To identify your volume device as an **AutoPlay** device, add an autorun.inf file to the root drive of your device. In the autorun.inf file, add a **CustomEvent** key to the **AutoRun** section. When your volume device connects to a PC, **AutoPlay** will find the autorun.inf file and treat your volume as a device. **AutoPlay** will create an **AutoPlay** event by using the name that you supplied for the **CustomEvent** key. You can then create an app and register the app as a handler for that **AutoPlay** event. When the device is connected to the PC, **AutoPlay** will show your app as a handler for your volume device. For more info on autorun.inf files, see [autorun.inf entries](#).

## Step 1: Create an autorun.inf file

In the root drive of your volume device, add a file named autorun.inf. Open the autorun.inf file and add the following text.

syntax

```
[AutoRun]
CustomEvent=AutoPlayCustomEventQuickstart
```

## Step 2: Create a new project and add AutoPlay declarations

1. Open Visual Studio and select **New Project** from the **File** menu. In the **Visual C#** section, under **Windows**, select **Blank App (Universal Windows)**. Name the application **AutoPlayCustomEvent** and click **OK**.
2. Open the Package.appxmanifest file and select the **Capabilities** tab. Select the **Removable Storage** capability. This gives the app access to the files and folders on removable storage devices.
3. In the manifest file, select the **Declarations** tab. In the **Available Declarations** drop-down list, select **AutoPlay Content** and click **Add**. Select the new **AutoPlay Content** item that was added to the **Supported Declarations** list.

### ⓘ Note

Alternatively, you can also choose to add an **AutoPlay Device** declaration for your custom AutoPlay event.

4. In the **Launch Actions** section for your **AutoPlay Content** event declaration, enter the values in the table below for the first launch action.

5. In the **Available Declarations** drop-down list, select **File Type Associations** and click **Add**. In the Properties of the new **File Type Associations** declaration, set the **Display Name** field to **Show .ms Files** and the **Name** field to **ms\_association**. In the **Supported File Types** section, click **Add New**. Set the **File Type** field to **.ms**. For content events, AutoPlay filters out any file types that aren't explicitly associated with your app.
6. Save and close the manifest file.

[Expand table](#)

| Setting             | Value                         |
|---------------------|-------------------------------|
| Verb                | show                          |
| Action Display Name | Show Files                    |
| Content Event       | AutoPlayCustomEventQuickstart |

The **Content Event** value is the text that you supplied for the **CustomEvent** key in your autorun.inf file. The **Action Display Name** setting identifies the string that AutoPlay displays for your app. The **Verb** setting identifies a value that is passed to your app for the selected option. You can specify multiple launch actions for an AutoPlay event and use the **Verb** setting to determine which option a user has selected for your app. You can tell which option the user selected by checking the **verb** property of the startup event arguments passed to your app. You can use any value for the **Verb** setting except, **open**, which is reserved.

## Step 3: Add the XAML UI

Open the MainPage.xaml file and add the following XAML to the default <Grid> section.

XAML

```
<StackPanel Orientation="Vertical">
    <TextBlock FontSize="28" Margin="10,0,800,0">Files</TextBlock>
    <TextBlock x:Name="FilesBlock" FontSize="22" Height="600"
Margin="10,0,800,0" />
</StackPanel>
```

## Step 4: Add the activation code

The code in this step calls a method to display the folders in the root drive of your volume device. For the AutoPlay content events, AutoPlay passes the root folder of the

storage device in the startup arguments passed to the application during the **OnFileActivated** event. You can retrieve this folder from the first element of the **Files** property.

Open the App.xaml.cs file and add the following code to the **App** class.

cs

```
protected override void OnFileActivated(FileActivatedEventArgs args)
{
    var rootFrame = Window.Current.Content as Frame;
    var page = rootFrame.Content as MainPage;

    // Call ShowFolders with root folder from device storage.
    page.DisplayFiles(args.Files[0] as Windows.Storage.StorageFolder);

    base.OnFileActivated(args);
}
```

#### ⓘ Note

The `DisplayFiles` method is added in the following step.

## Step 5: Add code to display folders

In the MainPage.xaml.cs file add the following code to the **MainPage** class.

cs

```
internal async void DisplayFiles(Windows.Storage.StorageFolder folder)
{
    foreach (Windows.Storage.StorageFile f in await ReadFiles(folder,
        ".ms"))
    {
        FilesBlock.Text += "  " + f.Name + "\n";
    }
}

internal async
System.Threading.Tasks.Task<IReadOnlyList<Windows.Storage.StorageFile>>
    ReadFiles(Windows.Storage.StorageFolder folder, string fileExtension)
{
    var options = new Windows.Storage.Search.QueryOptions();
    options.FileTypeFilter.Add(fileExtension);
    var query = folder.CreateFileQueryWithOptions(options);
    var files = await query.GetFilesAsync();
}
```

```
    return files;
}
```

## Step 6: Build and run the application

1. Press F5 to build and deploy the app (in debug mode).
2. To run your app, insert a memory card or another storage device into your PC.  
Then select your app from the list of AutoPlay handler options.

## AutoPlay event reference

The **AutoPlay** system allows apps to register for a variety of device and volume (disk) arrival events. To register for **AutoPlay** content events, you must enable the **Removable Storage** capability in your package manifest. This table shows the events that you can register for and when they are raised.

 Expand table

| Scenario                                              | Event                        | Description                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Using photos on a Camera                              | <b>WPD\ImageSource</b>       | Raised for cameras that are identified as Windows Portable Devices and offer the ImageSource capability.                                                                                                                                                                                        |
| Using music on an audio player                        | <b>WPD\AudioSource</b>       | Raised for media players that are identified as Windows Portable Devices and offer the AudioSource capability.                                                                                                                                                                                  |
| Using videos on a video camera                        | <b>WPD\VideoSource</b>       | Raised for video cameras that are identified as Windows Portable Devices and offer the VideoSource capability.                                                                                                                                                                                  |
| Access a connected flash drive or external hard drive | <b>StorageOnArrival</b>      | Raised when a drive or volume is connected to the PC. If the drive or volume contains a DCIM, AVCHD, or PRIVATE\ACHD folder in the root of the disk, the <b>ShowPicturesOnArrival</b> event is raised instead.                                                                                  |
| Using photos from mass storage (legacy)               | <b>ShowPicturesOnArrival</b> | Raised when a drive or volume contains a DCIM, AVCHD, or PRIVATE\ACHD folder in the root of the disk. If a user has enabled <b>Choose what to do with each type of media</b> in the AutoPlay Control Panel, AutoPlay will examine a volume connected to the PC to determine the type of content |

| Scenario                                               | Event                          | Description                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                        |                                | on the disk. When pictures are found, <b>ShowPicturesOnArrival</b> is raised.                                                                                                                                                                                                                                |
| Receiving photos with Proximity Sharing (tap and send) | <b>ShowPicturesOnArrival</b>   | When users send content with using proximity (tap and send), AutoPlay will examine the shared files to determine the type of content. If pictures are found, <b>ShowPicturesOnArrival</b> is raised.                                                                                                         |
| Using music from mass storage (legacy)                 | <b>PlayMusicFilesOnArrival</b> | If a user has enabled <b>Choose what to do with each type of media</b> in the AutoPlay Control Panel, AutoPlay will examine a volume connected to the PC to determine the type of content on the disk. When music files are found, <b>PlayMusicFilesOnArrival</b> is raised.                                 |
| Receiving music with Proximity Sharing (tap and send)  | <b>PlayMusicFilesOnArrival</b> | When users send content with using proximity (tap and send), AutoPlay will examine the shared files to determine the type of content. If music files are found, <b>PlayMusicFilesOnArrival</b> is raised.                                                                                                    |
| Using videos from mass storage (legacy)                | <b>PlayVideoFilesOnArrival</b> | If a user has enabled <b>Choose what to do with each type of media</b> in the AutoPlay Control Panel, AutoPlay will examine a volume connected to the PC to determine the type of content on the disk. When video files are found, <b>PlayVideoFilesOnArrival</b> is raised.                                 |
| Receiving videos with Proximity Sharing (tap and send) | <b>PlayVideoFilesOnArrival</b> | When users send content with using proximity (tap and send), AutoPlay will examine the shared files to determine the type of content. If video files are found, <b>PlayVideoFilesOnArrival</b> is raised.                                                                                                    |
| Handling mixed sets of files from a connected device   | <b>MixedContentOnArrival</b>   | If a user has enabled <b>Choose what to do with each type of media</b> in the AutoPlay Control Panel, AutoPlay will examine a volume connected to the PC to determine the type of content on the disk. If no specific content type is found (for example, pictures), <b>MixedContentOnArrival</b> is raised. |
| Handling mixed sets of files with Proximity            | <b>MixedContentOnArrival</b>   | When users send content with using proximity (tap and send), AutoPlay will examine the shared files to determine the type of content. If no specific content type                                                                                                                                            |

| Scenario                                     | Event                                                                                                                                   | Description                                                                                                                                                                                                                                 |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sharing (tap and send)                       |                                                                                                                                         | is found (for example, pictures), <b>MixedContentOnArrival</b> is raised.                                                                                                                                                                   |
| Handle video from optical media              | <b>PlayDVDMovieOnArrival</b><br><b>PlayBluRayOnArrival</b><br><b>PlayVideoCDMovieOnArrival</b><br><b>PlaySuperVideoCDMovieOnArrival</b> | When a disk is inserted into the optical drive, AutoPlay will examine the files to determine the type of content. When video files are found, the event corresponding to the type of optical disk is raised.                                |
| Handle music from optical media              | <b>PlayCDAudioOnArrival</b><br><b>PlayDVDAudioOnArrival</b>                                                                             | When a disk is inserted into the optical drive, AutoPlay will examine the files to determine the type of content. When music files are found, the event corresponding to the type of optical disk is raised.                                |
| Play enhanced disks                          | <b>PlayEnhancedCDOOnArrival</b><br><b>PlayEnhancedDVDOOnArrival</b>                                                                     | When a disk is inserted into the optical drive, AutoPlay will examine the files to determine the type of content. When an enhanced disk is found, the event corresponding to the type of optical disk is raised.                            |
| Handle writeable optical disks               | <b>HandleCDBurningOnArrival</b><br><b>HandleDVDBurningOnArrival</b><br><b>HandleBDBurningOnArrival</b>                                  | When a disk is inserted into the optical drive, AutoPlay will examine the files to determine the type of content. When a writable disk is found, the event corresponding to the type of optical disk is raised.                             |
| Handle any other device or volume connection | <b>UnknownContentOnArrival</b>                                                                                                          | Raised for all events in case content is found that does not match any of the AutoPlay content events. Use of this event is not recommended. You should only register your application for the specific AutoPlay events that it can handle. |

You can specify that AutoPlay raise a custom AutoPlay Content event using the **CustomEvent** entry in the autorun.inf file for a volume. For more info, see [Autorun.inf entries](#).

You can register your app as an AutoPlay Content or AutoPlay Device event handler by adding an extension to the package.appxmanifest file for your app. If you are using Visual Studio, you can add an **AutoPlay Content** or **AutoPlay Device** declaration in the **Declarations** tab. If you are editing the package.appxmanifest file for your app directly, add an **Extension** element to your package manifest that specifies either **windows.autoPlayContent** or **windows.autoPlayDevice** as the **Category**. For example,

the following entry in the package manifest adds an **AutoPlay Content** extension to register the app as a handler for the **ShowPicturesOnArrival** event.

XAML

```
<Applications>
  <Application Id="AutoPlayHandlerSample.App">
    <Extensions>
      <Extension Category="windows.autoPlayContent">
        <AutoPlayContent>
          <LaunchAction Verb="show" ActionDisplayName="Show Pictures"
            ContentEvent="ShowPicturesOnArrival" />
        </AutoPlayContent>
      </Extension>
    </Extensions>
  </Application>
</Applications>
```

---

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

# Reserved file and URI scheme names

Article • 10/20/2022

You can use URI associations to automatically launch your app when another app launches a specific URI scheme. But there are some URI associations that you can't use because they are reserved. If your app registers for a reserved association, that registration will be ignored. This topic lists the reserved file and URI scheme names that are not available to your app.

## Reserved file types

There are two types of reserved file types: file types reserved for built-in apps and file types reserved for the operating system. When a file type reserved for a built-in app is launched, only the built-in app will launch. Any attempt to register your app with that file type is ignored. Similarly, any attempt to register your app with a file type reserved for the operating system also will be ignored.

File types reserved for built-in apps

|       |          |       |        |
|-------|----------|-------|--------|
| .aac  | .icon    | .pem  | .wdp   |
| .aetx | .jpeg    | .png  | .wmv   |
| .asf  | .jxr     | .pptm | .xap   |
| .bmp  | .m4a     | .pptx | .xht   |
| .cer  | .m4r     | .qcp  | .xhtml |
| .dotm | .m4v     | .rtf  | .xltm  |
| .dotx | .mov     | .tif  | .xltx  |
| .gif  | .mp3     | .tiff | .xml   |
| .hdp  | .mp4     | .txt  | .xsl   |
| .htm  | .one     | .url  | .zip   |
| .html | .onetoc2 | .vcf  |        |
| .ico  | .p7b     | .wav  |        |

## File types reserved for the operating system

The following file types are reserved for the operating system

|                    |      |      |      |
|--------------------|------|------|------|
| .accountpicture-ms | its  | .ops | .url |
| .ade               | .jar | .pcd | .vb  |
| .adp               | .js  | .pif | .vbe |
| .app               | .jse | .pl  | .vbp |
| .appx              | .ksh | .plg | .vbs |
|                    |      |      |      |

|              |          |                |           |
|--------------|----------|----------------|-----------|
| .application | .lnk     | .plsc          | .vhd      |
| .appref-ms   | .mad     | .prf           | .vhdx     |
| .asp         | .maf     | .prg           | .vsmacros |
| .bas         | .mag     | .printerexport | .vsw      |
| .bat         | .mam     | .provxml       | .webpnp   |
| .cab         | .maq     | .ps1           | .ws       |
| .chm         | .mar     | .ps1xml        | .wsc      |
| .cmd         | .mas     | .ps2           | .wsf      |
| .cnt         | .mat     | .ps2xml        | .wsh      |
| .com         | .mau     | .psc1          | .xaml     |
| .cpf         | .mav     | .psc2          | .xdp      |
| .cpl         | .maw     | .psm1          | .xip      |
| .crd         | .mcf     | .pst           | .xnk      |
| .crds        | .mda     | .pvw           |           |
| .crt         | .mdb     | .py            |           |
| .csh         | .mde     | .pyc           |           |
| .der         | .mdt     | .pyo           |           |
| .dll         | .mdw     | .rb            |           |
| .drv         | .mdz     | .rbw           |           |
| .exe         | .msc     | .rdp           |           |
| .fxp         | .msh     | .reg           |           |
| .fon         | .msh1    | .rgu           |           |
| .gadget      | .msh1xml | .scf           |           |
| .grp         | .msh2    | .scr           |           |
| .hlp         | .msh2xml | .shb           |           |
| .hme         | .mshxml  | .shs           |           |
| .hpj         | .msi     | .sys           |           |
| .hta         | .msp     | .theme         |           |
| .inf         | .mst     | .tmp           |           |
| .ins         | .msu     | .tsk           |           |
| .isp         | .ocx     | .ttf           |           |

## Reserved URI scheme names

|                                             |                                   |                                   |                       |
|---------------------------------------------|-----------------------------------|-----------------------------------|-----------------------|
| application.manifest                        | inffile                           | ms-settings:network-dialup        | scrfile               |
| application.reference                       | insfile                           | ms-settings:network-ethernet      | scriptletfile         |
| batfile                                     | internetshortcut                  | ms-settings:network-mobilehotspot | shbfile               |
| bing                                        | javascript                        | ms-settings:network-proxy         | shcmdfile             |
| blob                                        | jscript                           | ms-settings:network-wifi          | shsfile               |
| callto                                      | jsefile                           | ms-settings:nfctransactions       | smb                   |
| cerfile                                     | ldap                              | ms-settings:notifications         | stickynotes           |
| chm.file                                    | lnkfile                           | ms-settings:personalization       | sysfile               |
| cmdfile                                     | mailto                            | ms-settings:privacy-calendar      | tel                   |
| comfile                                     | maps                              | ms-settings:privacy-contacts      | telnet                |
| cplfile                                     | microsoft.powershellscript.1      | ms-settings:privacy-customdevices | tn3270                |
| dllfile                                     | ms-accountpictureprovider         | ms-settings:privacy-feedback      | ttffile               |
| drvfile                                     | ms-appdata                        | ms-settings:privacy-location      | unknown               |
| dtmf                                        | ms-appx                           | ms-settings:privacy-messaging     | usertileprovider      |
| exefile                                     | ms-autoplay                       | ms-settings:privacy-microphone    | vbefile               |
| explorer.assocactionid.burnselection        | ms-excel                          | ms-settings:privacy-speechtyping  | vbscript              |
| explorer.assocactionid.closesession         | msi.package                       | ms-settings:privacy-webcam        | vbsfile               |
| explorer.assocactionid.erasedisc            | msi.patch                         | ms-settings:proximity             | wallet                |
| explorer.assocactionid.zipselection         | ms-powerpoint                     | ms-settings:regionlanguage        | windows.gadget        |
| explorer.assocprotocol.search-ms            | ms-settings                       | ms-settings:screenrotation        | windowsmediacenterapp |
| explorer.burnselection                      | ms-settings:batterysaver          | ms-settings:speech                | windowsmediacenterssl |
| explorer.burnselectionexplorer.closesession | ms-settings:batterysaver-settings | ms-settings:storagesense          | windowsmediacenterweb |
|                                             |                                   |                                   |                       |

|                       |                                           |                               |                         |
|-----------------------|-------------------------------------------|-------------------------------|-------------------------|
| explorer.closesession | ms-settings:batterysaver-<br>usagedetails | ms-<br>settings:windowsupdate | wmp11.assocprotocol.mms |
| explorer.erasedisc    | ms-settings:bluetooth                     | ms-settings:workplace         | wsffile                 |
| explorer.zipselection | ms-<br>settings:connecteddevices          | ms-windows-store              | wsfile                  |
| file                  | ms-settings:cortanasearch                 | ms-word                       | wshfile                 |
| fonfile               | ms-settings:datasense                     | ocxfile                       | xbls                    |
| hlpfile               | ms-settings:dateandtime                   | office                        | zune                    |
| htafile               | ms-settings:display                       | onenote                       |                         |
| http                  | ms-settings:lockscreen                    | piffile                       |                         |
| https                 | ms-settings:maps                          | regfile                       |                         |
| iehistory             | ms-settings:network-<br>airplanemode      | res                           |                         |
| ierss                 | ms-settings:network-<br>cellular          | rlogin                        |                         |

# Use app services and extensions

Article • 10/20/2022

This section describes how to integrate app services into your UWP app to allow the sharing of data and functionality across apps.

| Topic                                                                             | Description                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Create and consume an app service</a>                                 | Learn how to write a Universal Windows Platform (UWP) app that can provide services to other UWP apps and how to consume those services.                                                          |
| <a href="#">Convert an app service to run in the same process as its host app</a> | Convert app service code that ran in a separate background process into code that runs inside the same process as your app service provider.                                                      |
| <a href="#">Extend your app with app services, extensions, and packages</a>       | Determine which technology to use to extend and componentize your app and get a brief overview of each.                                                                                           |
| <a href="#">Create and host an app extension</a>                                  | Write and host Universal Windows Platform (UWP) app extensions to extend your app via packages that users can install from the Microsoft Store.                                                   |
| <a href="#">Create hosted apps</a>                                                | Create a packaged component (such as an app or a script file) that behaves like a standalone Windows 10 app but requires a separate host app (a UWP or packaged desktop app) in order to execute. |

## Related topics

- [Communicate with a remote app service](#)
- [Universal Windows Platform \(UWP\) app samples](#) 

# Create and consume an app service

Article • 01/04/2023

## Important

The code listings in this topic are C# only. For an app service sample app in C++/WinRT as well as C#, see [App service sample app](#) .

App services are UWP apps that provide services to other UWP apps. They are analogous to web services, on a device. An app service runs as a background task in the host app and can provide its service to other apps. For example, an app service might provide a bar code scanner service that other apps could use. Or perhaps an Enterprise suite of apps has a common spell checking app service that is available to the other apps in the suite. App services let you create UI-less services that apps can call on the same device, and starting with Windows 10, version 1607, on remote devices.

Starting in Windows 10, version 1607, you can create app services that run in the same process as the host app. This article focuses on creating and consuming an app service that runs in a separate background process. See [Convert an app service to run in the same process as its host app](#) for more details about running an app service in the same process as the provider.

## Create a new app service provider project

In this how-to, we'll create everything in one solution for simplicity.

1. In Visual Studio 2015 or later, create a new UWP app project and name it **AppServiceProvider**.
  - a. Select **File > New > Project...**
  - b. In the **Create a new project** dialog box, select **Blank App (Universal Windows) C#**. This will be the app that makes the app service available to other UWP apps.
  - c. Click **Next**, and then name the project **AppServiceProvider**, choose a location for it, and then click **Create**.
2. When asked to select a **Target** and **Minimum version** for the project, select at least **10.0.14393**. If you want to use the new **SupportsMultipleInstances** attribute, you must be using Visual Studio 2017 or Visual Studio 2019, and target **10.0.15063 (Windows 10 Creators Update)** or later.

# Add an app service extension to Package.appxmanifest

In the **AppServiceProvider** project, open the **Package.appxmanifest** file in a text editor:

1. Right-click it in the **Solution Explorer**.
2. Select **Open With**.
3. Select **XML (Text) Editor**.

Add the following **AppService** extension inside the **<Application>** element. This example advertises the **com.microsoft.inventory** service and is what identifies this app as an app service provider. The actual service will be implemented as a background task. The app service project exposes the service to other apps. We recommend using a reverse domain name style for the service name.

Note that the **xmlns:uap4** namespace prefix and the **uap4:SupportsMultipleInstances** attribute are only valid if you are targeting Windows SDK version 10.0.15063 or later. You can safely remove them if you are targeting older SDK versions.

## ⓘ Note

For an app service sample app in C++/WinRT as well as C#, see [App service sample app](#).

XML

```
<Package
...
  xmlns:uap3="http://schemas.microsoft.com/appx/manifest/uap/windows10/3"
  xmlns:uap4="http://schemas.microsoft.com/appx/manifest/uap/windows10/4"
...
  <Applications>
    <Application Id="AppServiceProvider.App"
      Executable="$targetnametoken$.exe"
      EntryPoint="AppServiceProvider.App">
      ...
      <Extensions>
        <uap:Extension Category="windows.appService"
          EntryPoint="MyAppService.Inventory">
          <uap3:AppService Name="com.microsoft.inventory"
            uap4:SupportsMultipleInstances="true"/>
          </uap:Extension>
        </Extensions>
      ...
    </Application>
  </Applications>
```

The `Category` attribute identifies this application as an app service provider.

The `EntryPoint` attribute identifies the namespace qualified class that implements the service, which we'll implement next.

The `SupportsMultipleInstances` attribute indicates that each time the app service is called that it should run in a new process. This is not required but is available to you if you need that functionality and are targeting the 10.0.15063 SDK (**Windows 10 Creators Update**) or later. It also should be prefaced by the `uap4` namespace.

## Create the app service

1. An app service can be implemented as a background task. This enables a foreground application to invoke an app service in another application. To create an app service as a background task, add a new Windows Runtime component project to the solution (**File > Add > New Project**) named **MyAppService**. In the **Add New Project** dialog box, choose **Installed > Visual C# > Windows Runtime Component (Universal Windows)**.
2. In the **AppServiceProvider** project, add a project-to-project reference to the new **MyAppService** project (in the **Solution Explorer**, right-click on the **AppServiceProvider** project > **Add > Reference > Projects > Solution**, select **MyAppService > OK**). This step is critical because if you do not add the reference, the app service won't connect at runtime.
3. In the **MyAppService** project, add the following **using** statements to the top of **Class1.cs**:

```
C#  
  
using Windows.ApplicationModel.AppService;  
using Windows.ApplicationModel.Background;  
using Windows.Foundation.Collections;
```

4. Rename **Class1.cs** to **Inventory.cs**, and replace the stub code for **Class1** with a new background task class named **Inventory**:

```
C#  
  
public sealed class Inventory : IBackgroundTask  
{  
    private BackgroundTaskDeferral backgroundTaskDeferral;  
    private AppServiceConnection appServiceconnection;
```

```

    private String[] inventoryItems = new string[] { "Robot vacuum",
"Chair" };
    private double[] inventoryPrices = new double[] { 129.99, 88.99 };

    public void Run(IBackgroundTaskInstance taskInstance)
    {
        // Get a deferral so that the service isn't terminated.
        this.backgroundTaskDeferral = taskInstance.GetDeferral();

        // Associate a cancellation handler with the background task.
        taskInstance.Canceled += OnTaskCanceled;

        // Retrieve the app service connection and set up a listener
        for incoming app service requests.
        var details = taskInstance.TriggerDetails as
AppServiceTriggerDetails;
        appServiceconnection = details.AppServiceConnection;
        appServiceconnection.RequestReceived += OnRequestReceived;
    }

    private async void OnRequestReceived(AppServiceConnection sender,
AppServiceRequestReceivedEventArgs args)
    {
        // This function is called when the app service receives a
        request.
    }

    private void OnTaskCanceled(IBackgroundTaskInstance sender,
BackgroundTaskCancellationReason reason)
    {
        if (this.backgroundTaskDeferral != null)
        {
            // Complete the service deferral.
            this.backgroundTaskDeferral.Complete();
        }
    }
}

```

This class is where the app service will do its work.

**Run** is called when the background task is created. Because background tasks are terminated after **Run** completes, the code takes out a deferral so that the background task will stay up to serve requests. An app service that is implemented as a background task will stay alive for about 30 seconds after it receives a call unless it is called again within that time window or a deferral is taken out. If the app service is implemented in the same process as the caller, the lifetime of the app service is tied to the lifetime of the caller.

The lifetime of the app service depends on the caller:

- If the caller is in the foreground, the app service lifetime is the same as the caller.
- If the caller is in the background, the app service gets 30 seconds to run. Taking out a deferral provides an additional one time 5 seconds.

**OnTaskCanceled** is called when the task is canceled. The task is canceled when the client app disposes the [AppServiceConnection](#), the client app is suspended, the OS is shut down or sleeps, or the OS runs out of resources to run the task.

## Write the code for the app service

**OnRequestReceived** is where the code for the app service goes. Replace the stub **OnRequestReceived** in **MyAppService's Inventory.cs** with the code from this example. This code gets an index for an inventory item and passes it, along with a command string, to the service to retrieve the name and the price of the specified inventory item. For your own projects, add error handling code.

C#

```
private async void OnRequestReceived(AppServiceConnection sender,
AppServiceRequestReceivedEventArgs args)
{
    // Get a deferral because we use an awaitable API below to respond to
    the message
    // and we don't want this call to get canceled while we are waiting.
    var messageDeferral = args.GetDeferral();

    ValueSet message = args.Request.Message;
    ValueSet returnData = new ValueSet();

    string command = message["Command"] as string;
    int? inventoryIndex = message["ID"] as int?;

    if (inventoryIndex.HasValue &&
        inventoryIndex.Value >= 0 &&
        inventoryIndex.Value < inventoryItems.GetLength(0))
    {
        switch (command)
        {
            case "Price":
            {
                returnData.Add("Result",
inventoryPrices[inventoryIndex.Value]);
                returnData.Add("Status", "OK");
                break;
            }

            case "Item":
            {
```

```

        returnData.Add("Result",
inventoryItems[inventoryIndex.Value]);
        returnData.Add("Status", "OK");
        break;
    }

    default:
    {
        returnData.Add("Status", "Fail: unknown command");
        break;
    }
}
}
else
{
    returnData.Add("Status", "Fail: Index out of range");
}

try
{
    // Return the data to the caller.
    await args.Request.SendResponseAsync(returnData);
}
catch (Exception e)
{
    // Your exception handling code here.
}
finally
{
    // Complete the deferral so that the platform knows that we're done
    responding to the app service call.
    // Note for error handling: this must be called even if
    SendResponseAsync() throws an exception.
    messageDeferral.Complete();
}
}

```

Note that **OnRequestReceived** is **async** because we make an awaitable method call to **SendResponseAsync** in this example.

A deferral is taken so that the service can use **async** methods in the **OnRequestReceived** handler. It ensures that the call to **OnRequestReceived** does not complete until it is done processing the message. **SendResponseAsync** sends the result to the caller. **SendResponseAsync** does not signal the completion of the call. It is the completion of the deferral that signals to **SendMessageAsync** that **OnRequestReceived** has completed. The call to **SendResponseAsync** is wrapped in a try/finally block because you must complete the deferral even if **SendResponseAsync** throws an exception.

App services use **ValueSet** objects to exchange information. The size of the data you may pass is only limited by system resources. There are no predefined keys for you to

use in your **ValueSet**. You must determine which key values you will use to define the protocol for your app service. The caller must be written with that protocol in mind. In this example, we have chosen a key named **Command** that has a value that indicates whether we want the app service to provide the name of the inventory item or its price. The index of the inventory name is stored under the **ID** key. The return value is stored under the **Result** key.

An **AppServiceClosedStatus** enum is returned to the caller to indicate whether the call to the app service succeeded or failed. An example of how the call to the app service could fail is if the OS aborts the service endpoint because its resources have been exceeded. You can return additional error information via the **ValueSet**. In this example, we use a key named **Status** to return more detailed error information to the caller.

The call to **SendResponseAsync** returns the **ValueSet** to the caller.

## Deploy the service app and get the package family name

The app service provider must be deployed before you can call it from a client. You can deploy it by selecting **Build > Deploy Solution** in Visual Studio.

You will also need the package family name of the app service provider in order to call it. You can get it by opening the **AppServiceProvider** project's **Package.appxmanifest** file in the designer view (double-click it in the **Solution Explorer**). Select the **Packaging** tab, copy the value next to **Package family name**, and paste it somewhere like Notepad for now.

## Write a client to call the app service

1. Add a new blank Windows Universal app project to the solution with **File > Add > New Project**. In the **Add New Project** dialog box, choose **Installed > Visual C# > Blank App (Universal Windows)** and name it **ClientApp**.
2. In the **ClientApp** project, add the following **using** statement to the top of **MainPage.xaml.cs**:

```
C#  
  
using Windows.ApplicationModel.AppService;
```

3. Add a text box called **textBox** and a button to **MainPage.xaml**.

4. Add a button click handler for the button called **button\_Click**, and add the keyword **async** to the button handler's signature.
5. Replace the stub of your button click handler with the following code. Be sure to include the `inventoryService` field declaration.

C#

```
private AppServiceConnection inventoryService;

private async void button_Click(object sender, RoutedEventArgs e)
{
    // Add the connection.
    if (this.inventoryService == null)
    {
        this.inventoryService = new AppServiceConnection();

        // Here, we use the app service name defined in the app service
        // provider's Package.appxmanifest file in the <Extension>
        section.
        this.inventoryService.AppServiceName =
"com.microsoft.inventory";

        // Use Windows.ApplicationModel.Package.Current.Id.FamilyName
        // within the app service provider to get this value.
        this.inventoryService.PackageFamilyName = "Replace with the
package family name";

        var status = await this.inventoryService.OpenAsync();

        if (status != AppServiceConnectionStatus.Success)
        {
            textBox.Text= "Failed to connect";
            this.inventoryService = null;
            return;
        }
    }

    // Call the service.
    int idx = int.Parse(textBox.Text);
    var message = new ValueSet();
    message.Add("Command", "Item");
    message.Add("ID", idx);
    AppServiceResponse response = await
this.inventoryService.SendMessageAsync(message);
    string result = "";

    if (response.Status == AppServiceResponseStatus.Success)
    {
        // Get the data that the service sent to us.
        if (response.Message["Status"] as string == "OK")
        {
            result = response.Message["Result"] as string;
        }
    }
}
```

```

    }
}

message.Clear();
message.Add("Command", "Price");
message.Add("ID", idx);
response = await this.inventoryService.SendMessageAsync(message);

if (response.Status == AppServiceResponseStatus.Success)
{
    // Get the data that the service sent to us.
    if (response.Message["Status"] as string == "OK")
    {
        result += " : Price = " + response.Message["Result"] as
string;
    }
}

textBox.Text = result;
}

```

Replace the package family name in the line

`this.inventoryService.PackageFamilyName = "Replace with the package family name";` with the package family name of the **AppServiceProvider** project that you obtained above in [Deploy the service app and get the package family name](#).

#### ⓘ Note

Make sure to paste in the string literal, rather than putting it in a variable. It will not work if you use a variable.

The code first establishes a connection with the app service. The connection will remain open until you dispose `this.inventoryService`. The app service name must match the `AppService` element's `Name` attribute that you added to the **AppServiceProvider** project's **Package.appxmanifest** file. In this example, it is `<uap3:AppService Name="com.microsoft.inventory"/>`.

A **ValueSet** named `message` is created to specify the command that we want to send to the app service. The example app service expects a command to indicate which of two actions to take. We get the index from the text box in the client app, and then call the service with the `Item` command to get the description of the item. Then, we make the call with the `Price` command to get the item's price. The button text is set to the result.

Because `AppServiceResponseStatus` only indicates whether the operating system was able to connect the call to the app service, we check the `Status` key in the `ValueSet` we receive from the app service to ensure that it was able to fulfill the request.

6. Set the **ClientApp** project to be the startup project (right-click it in the **Solution Explorer** > **Set as StartUp Project**) and run the solution. Enter the number 1 into the text box and click the button. You should get "Chair : Price = 88.99" back from the service.



If the app service call fails, check the following in the **ClientApp** project:

1. Verify that the package family name assigned to the inventory service connection matches the package family name of the **AppServiceProvider** app. See the line in **button\_Click** with `this.inventoryService.PackageFamilyName = "...";`.
2. In **button\_Click**, verify that the app service name that is assigned to the inventory service connection matches the app service name in the **AppServiceProvider's Package.appxmanifest** file. See: `this.inventoryService.AppServiceName = "com.microsoft.inventory";`.
3. Ensure that the **AppServiceProvider** app has been deployed. (In the **Solution Explorer**, right-click the solution and choose **Deploy Solution**).

## Debug the app service

1. Ensure that the solution is deployed before debugging because the app service provider app must be deployed before the service can be called. (In Visual Studio, **Build** > **Deploy Solution**).
2. In the **Solution Explorer**, right-click the **AppServiceProvider** project and choose **Properties**. From the **Debug** tab, change the **Start action** to **Do not launch, but debug my code when it starts**. (Note, if you were using C++ to implement your app service provider, from the **Debugging** tab you would change **Launch Application** to **No**).

3. In the **MyAppService** project, in the **Inventory.cs** file, set a breakpoint in **OnRequestReceived**.
4. Set the **AppServiceProvider** project to be the startup project and press **F5**.
5. Start **ClientApp** from the Start menu (not from Visual Studio).
6. Enter the number 1 into the text box and press the button. The debugger will stop in the app service call on the breakpoint in your app service.

## Debug the client

1. Follow the instructions in the preceding step to debug the client that calls the app service.
2. Launch **ClientApp** from the Start menu.
3. Attach the debugger to the **ClientApp.exe** process (not the **ApplicationFrameHost.exe** process). (In Visual Studio, choose **Debug > Attach to Process....**)
4. In the **ClientApp** project, set a breakpoint in **button\_Click**.
5. The breakpoints in both the client and the app service will now be hit when you enter the number 1 into the text box of **ClientApp** and click the button.

## General app service troubleshooting

If you encounter an **AppUnavailable** status after trying to connect to an app service, check the following:

- Ensure that the app service provider project and app service project are deployed. Both need to be deployed before running the client because otherwise the client won't have anything to connect to. You can deploy from Visual Studio by using **Build > Deploy Solution**.
- In the **Solution Explorer**, ensure that your app service provider project has a project-to-project reference to the project that implements the app service.
- Verify that the `<Extensions>` entry, and its child elements, have been added to the **Package.appxmanifest** file belonging to the app service provider project as specified above in [Add an app service extension to Package.appxmanifest](#).
- Ensure that the `AppServiceConnection.AppServiceName` string in your client that calls the app service provider matches the `<uap3:AppService Name="..." />` specified in the app service provider project's **Package.appxmanifest** file.
- Ensure that the `AppServiceConnection.PackageFamilyName` matches the package family name of the app service provider component as specified above in [Add an app service extension to Package.appxmanifest](#)

- For out-of-proc app services such as the one in this example, validate that the `EntryPoint` specified in the `<uap:Extension ...>` element of your app service provider project's **Package.appxmanifest** file matches the namespace and class name of the public class that implements [IBackgroundTask](#) in your app service project.

## Troubleshoot debugging

If the debugger does not stop at breakpoints in your app service provider or app service projects, check the following:

- Ensure that the app service provider project and app service project are deployed. Both need to be deployed before running the client. You can deploy them from Visual Studio by using **Build > Deploy Solution**.
- Ensure that the project you want to debug is set as the startup project and that the debugging properties for that project are set to not run the project when **F5** is pressed. Right-click the project, then click **Properties**, and then **Debug** (or **Debugging** in C++). In C#, change the **Start action** to **Do not launch, but debug my code when it starts**. In C++, set **Launch Application** to **No**.

## Remarks

This example provides an introduction to creating an app service that runs as a background task and calling it from another app. The key things to note are:

- Create a background task to host the app service.
- Add the `windows.appService` extension to the app service provider's **Package.appxmanifest** file.
- Obtain the package family name of the app service provider so that we can connect to it from the client app.
- Add a project-to-project reference from the app service provider project to the app service project.
- Use [Windows.ApplicationModel.AppService.AppServiceConnection](#) to call the service.

## Full code for MyAppService

C#

```
using System;
using Windows.ApplicationModel.AppService;
```